

## Attività svolta: DataGridView, Array e UDT

### Query e report:

Il comando **SELECT** recupera le righe eventualmente filtrate e ordinate.

**WHERE** filtra (solo allarmi di oggi), **ORDER BY** ordina (dal più recente), **LIMIT** limita (solo ultimi 10).

```
1 riferimento
65 public int ConteggioTuttiAllarmi()
66 {
67     var conn = new SqlConnection(_connString);
68     conn.Open();
69     var cmd = conn.CreateCommand();
70     cmd.CommandText = "SELECT COUNT(*) FROM Allarmi;";
71     return Convert.ToInt32(cmd.ExecuteScalar());
72 }
73
74 1 riferimento
75 public int ConteggioAllarmiOggi()
76 {
77     var conn = new SqlConnection(_connString);
78     conn.Open();
79     var cmd = conn.CreateCommand();
80     cmd.CommandText = "SELECT COUNT(*) FROM Allarmi WHERE Timestamp >= date('now');";
81     return Convert.ToInt32(cmd.ExecuteScalar());
82 }
83 1 riferimento
84 public double GiriMedi()
85 {
86     var conn = new SqlConnection(_connString);
87     conn.Open();
88     var cmd = conn.CreateCommand();
89     cmd.CommandText = "SELECT AVG(Giri) FROM Produzione WHERE Motore = 1;";
90
91     var risultato = cmd.ExecuteScalar();
92     if (risultato == DBNull.Value || risultato == null) return 0.0;
93
94     return Convert.ToDouble(risultato);
95 }
```

```
73 private void RiepilogoDataGridView()
74 {
75     if (plc == null || !plc.IsConnected) return;
76     try
77     {
78         dataGridAllarmi.DataSource = null;
79
80         int totali = repos.ConteggioTuttiAllarmi();
81         int oggi = repos.ConteggioAllarmiOggi();
82         double mediaGiri = repos.GiriMedi();
83
84         var sorgenteDati = new List<RigaRiepilogo>
85         {
86             new RigaRiepilogo
87             {
88                 AllarmiTotali = totali,
89                 AllarmiOggi = oggi,
90                 GiriMediMotoreOn = Math.Round(mediaGiri, 1)
91             }
92         };
93
94         dataGridAllarmi.DataSource = sorgenteDati;
95
96         dataGridAllarmi.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;
97     }
98     catch (Exception ex)
99     {
100         Console.WriteLine($"Errore: riepilogo DataGridView: {ex.Message}");
101     }
102 }
```

## Array e UDT:

Un **Array** è una sequenza di elementi dello stesso tipo e si dichiara in un DataBlock così:

| NOME   | TIPO               |
|--------|--------------------|
| Misure | Array[0..9] of Int |

| DB_Array |             |                    |        |                 |                          |                                     |                                     |                                     |                          |    |
|----------|-------------|--------------------|--------|-----------------|--------------------------|-------------------------------------|-------------------------------------|-------------------------------------|--------------------------|----|
|          | Nome        | Tipo di dati       | Offset | Valore di avvio | Ritenzione               | Accessibile ...                     | Scrivi...                           | Visibile in ...                     | Valore di i...           | Co |
| 1        | ▼ Static    |                    |        |                 | <input type="checkbox"/> | <input type="checkbox"/>            | <input type="checkbox"/>            | <input type="checkbox"/>            | <input type="checkbox"/> |    |
| 2        | ■ Misure    | Array[0..9] of Int | ...    |                 | <input type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input type="checkbox"/> |    |
| 3        | ■ Misure[0] | Int                | ...    | 0               | <input type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input type="checkbox"/> |    |
| 4        | ■ Misure[1] | Int                | ...    | 0               | <input type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input type="checkbox"/> |    |
| 5        | ■ Misure[2] | Int                | ...    | 0               | <input type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input type="checkbox"/> |    |
| 6        | ■ Misure[3] | Int                | ...    | 0               | <input type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input type="checkbox"/> |    |
| 7        | ■ Misure[4] | Int                | ...    | 0               | <input type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input type="checkbox"/> |    |
| 8        | ■ Misure[5] | Int                | ...    | 0               | <input type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input type="checkbox"/> |    |
| 9        | ■ Misure[6] | Int                | ...    | 0               | <input type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input type="checkbox"/> |    |
| 10       | ■ Misure[7] | Int                | ...    | 0               | <input type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input type="checkbox"/> |    |
| 11       | ■ Misure[8] | Int                | ...    | 0               | <input type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input type="checkbox"/> |    |
| 12       | ■ Misure[9] | Int                | ...    | 0               | <input type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input type="checkbox"/> |    |

Un **UDT (User Defined Type)** è un tipo di dato personalizzato che raggruppa più variabili come una struct/classe:

| udtRicetta |                                                                                                  |                                                                                     |                   |                                                                                     |                                                                                     |                                                                                       |                                                                                       |           |     |
|------------|--------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|-----------|-----|
|            | Nome                                                                                             | Tipo di dati                                                                        | Valore di default | Accessibile ...                                                                     | Scrivi...                                                                           | Visibile in ..                                                                        | Valore di i...                                                                        | Controllo | Com |
| 1          |  ▼ Ricetta    | Struct                                                                              |                   |  |  |  |  |           |     |
| 2          |  ■ Nome       | String[20]                                                                          | ''                |  |  |  |  |           |     |
| 3          |  ■ GiniTarget | Int                                                                                 | 0                 |  |  |  |  |           |     |
| 4          |  ■ DurataSec  | Int                                                                                 | 0                 |  |  |  |  |           |     |
| 5          |  ■ Attiva     | Bool                                                                                | false             |  |  |  |  |           |     |
| 5          | ■ <Aggiungi>                                                                                     |                                                                                     |                   |  |  |  |  |           |     |
| 7          | <Aggiungi>                                                                                       |  |                   |  |  |  |  |           |     |

| DB_Array |                      |                    |        |                 |                          |                                     |                                     |                                     |                                     |           |          |
|----------|----------------------|--------------------|--------|-----------------|--------------------------|-------------------------------------|-------------------------------------|-------------------------------------|-------------------------------------|-----------|----------|
|          | Nome                 | Tipo di dati       | Offset | Valore di avvio | Ritenzione               | Accessibile ...                     | Scrivi...                           | Visibile in ...                     | Valore di i...                      | Controllo | Commento |
| 1        | ▼ Static             |                    |        |                 | <input type="checkbox"/> | <input type="checkbox"/>            | <input type="checkbox"/>            | <input type="checkbox"/>            | <input type="checkbox"/>            |           |          |
| 2        | ▣ ▸ Misure           | Array[0..9] of Int | ...    |                 | <input type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input type="checkbox"/>            |           |          |
| 3        | ▣ ▼ Ricetta Corrente | "udtRicetta"       | ...    |                 | <input type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input type="checkbox"/>            |           |          |
| 4        | ▣ ▸ Ricetta          | Struct             | ...    |                 | <input type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input type="checkbox"/>            |           |          |
| 5        | ▣ ▸ Nome             | String[20]         | ...    | ''              | <input type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input type="checkbox"/>            |           |          |
| 6        | ▣ ▸ GiniTarget       | Int                | ...    | 0               | <input type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input type="checkbox"/>            |           |          |
| 7        | ▣ ▸ DurataSec        | Int                | ...    | 0               | <input type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input type="checkbox"/>            |           |          |
| 8        | ▣ ▸ Attiva           | Bool               | ...    | false           | <input type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input type="checkbox"/>            |           |          |
| 9        | ▣ ▼ ElencoRicette    | Array[0..4] of ... | ...    |                 | <input type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |           |          |
| 10       | ▣ ▸ ElencoRicette[0] | "udtRicetta"       | ...    |                 | <input type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |           |          |
| 11       | ▣ ▸ ElencoRicette[1] | "udtRicetta"       | ...    |                 | <input type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |           |          |
| 12       | ▣ ▸ ElencoRicette[2] | "udtRicetta"       | ...    |                 | <input type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |           |          |
| 13       | ▣ ▸ ElencoRicette[3] | "udtRicetta"       | ...    |                 | <input type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |           |          |
| 14       | ▣ ▸ ElencoRicette[4] | "udtRicetta"       | ...    |                 | <input type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |           |          |
| 15       | ▣ ▸ <Aggiungi>       |                    | ...    |                 | <input type="checkbox"/> | <input type="checkbox"/>            | <input type="checkbox"/>            | <input type="checkbox"/>            | <input type="checkbox"/>            |           |          |

## Leggere un UDT in C#:

Si legge il blocco di byte dell'UDT e si mappano i campi su una classe C#. Per un array di UDT per leggere l'elemento i-esimo, l'offset è:

$\text{offsetBase} + i * (\text{dimensione UDT})$

```
11 public class Ricetta
12 {
13     2 riferimenti
14     public string Nome { get; set; }
15     1 riferimento
16     public short GiriTarget { get; set; }
17     1 riferimento
18     public short DurataSec { get; set; }
19     1 riferimento
20     public bool Attiva { get; set; }
21
22     1 riferimento
23     public Ricetta LeggiRicetta(int dbNum, int offBase)
24     {
25         // Leggiamo TUTTA la ricetta (28 byte) in un colpo solo
26         byte[] b = Form1.plc.ReadBytes(S7.Net.DataType.DataBlock, dbNum, offBase, 28);
27
28         if (b == null || b.Length < 28)
29             return new Ricetta { Nome = "Errore Lettura" };
30
31         // Il nome occupa i primi 22 byte (dal byte 0 al byte 21)
32         string nomeEstratto = System.Text.Encoding.ASCII.GetString(b, 0, 20).Trim(); // Occupa da bit 0 a bit 21
33
34         // I dati numerici si trovano ESATTAMENTE a questi indici dell'array caricato:
35         // GiriTarget è all'offset 22 e 23 (corrisponde a 70.0 nel PLC)
36         // DurataSec è all'offset 24 e 25 (corrisponde a 72.0 nel PLC)
37         // Attiva è all'offset 26 (corrisponde a 74.0 nel PLC)
38
39         return new Ricetta
40         {
41             Nome = nomeEstratto,
42             GiriTarget = (short)S7.Net.Types.Int.FromByteArray(new byte[] { b[22], b[23] }), // Occupa bit 22 e 23
43             DurataSec = (short)S7.Net.Types.Int.FromByteArray(new byte[] { b[24], b[25] }), // Occupa bit 24 e 25
44             Attiva = (b[26] & 0x01) != 0 // Occupa bit 26
45         };
46     }
47 }
```

Se voglio mostrare le ricette (in questo caso 3) in un dataGrid:

```
103 private void MostraRicette()
104 {
105     if (plc == null || !plc.IsConnected) return;
106
107     try
108     {
109         List<Ricetta> listaRicette = new List<Ricetta>();
110
111         for (int i = 0; i < 3; i++)
112         {
113             int offsetRicettaCorrente = 48 + (i * 28);
114
115             Ricetta r = new Ricetta();
116             r = r.LeggiRicetta(20, offsetRicettaCorrente);
117
118             listaRicette.Add(r);
119         }
120
121         dataGridRicetta.DataSource = null;
122         dataGridRicetta.DataSource = listaRicette;
123
124         dataGridRicetta.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;
125
126         dataGridRicetta.ReadOnly = true;
127     }
128     catch (Exception ex)
129     {
130         MessageBox.Show($"Errore nel caricamento delle ricette nella griglia: {ex.Message}");
131     }
132 }
```

## Organizzazione del codice:

Man mano che cresce il progetto, gli indirizzi offset S7.NET sparsi nel codice diventano un problema: se si cambia un offset nel PLC bisogna cercarli tutti o se ne sbaglia uno si ha poi un bug difficile da trovare. Per prima cosa raccogliamo tutti gli indirizzi in un unico punto come costanti, così se cambio un offset basta correggerlo in un unico posto...

```
10 namespace Mini_SupervisoreC_
11 {
12     25 riferimenti
13     public static class IndirizziDB
14     {
15         public const int DB = 12;
16
17         public const string ComandoMarcia = "DB12.DBX0.0";
18         public const string ComandoReset = "DB12.DBX0.1";
19         public const string SetpointGiri = "DB12.DBW2";
20         public const string GiriAttuali = "DB12.DBW4";
21         public const string MotoreAttivo = "DB12.DBX6.0";
22         public const string AllarmeSovragiri = "DB12.DBX6.1";
23         public const string OreFunzionamento = "DB12.DBD8";
24
25         0 riferimenti
26         public static LineaStato LeggiStatoCompleto()
27         {
28             // Leggiamo i 12 byte del DB12 partendo dall'offset 0
29             byte[] b = Form1.plc.ReadBytes(S7.Net.DataType.DataBlock, IndirizziDB.DB, 0, 12);
30
31             return new LineaStato
32             {
33                 // DBX 0.0 -> Primo byte, bit 0
34                 ComandoMarcia = (b[0] & 0x01) != 0,
35
36                 // DBW 2 -> Byte 2 e 3 (Usiamo l'inversione nativa di S7.Net per evitare sfasamenti)
37                 SetpointGiri = (short)S7.Net.Types.Int.FromByteArray(new byte[] { b[2], b[3] }),
38
39                 // DBW 4 -> Byte 4 e 5
40                 GiriAttuali = (short)S7.Net.Types.Int.FromByteArray(new byte[] { b[4], b[5] }),
41
42                 // DBX 6.0 -> Byte 6, bit 0
43                 MotoreAttivo = (b[6] & 0x01) != 0,
44
45                 // DBX 6.1 -> Byte 6, bit 1
46                 AllarmeSovragiri = (b[6] & 0x02) != 0,
47
48                 // DBD 8 -> Passiamo i 4 byte corretti (dall'8 all'11) per il tipo Real (float)
49                 OreFunzionamento = S7.Net.Types.Real.FromByteArray(new byte[] { b[8], b[9], b[10], b[11] })
50             };
51         }
52     }
53 }
```

La funzione **LeggiStatoCompleto()** serve per leggere tutto il Data Block in una volta sola. Quando i dati crescono, fare tante letture separate diventa inefficiente...

```
31     0 riferimenti
32     public void Avvia() => _plc.Write(IndirizziDB.ComandoMarcia, true);
33     0 riferimenti
34     public void Ferma() => _plc.Write(IndirizziDB.ComandoMarcia, false);
35     0 riferimenti
36     public void impostaSetpoint(short RPM) => _plc.Write(IndirizziDB.SetpointGiri, RPM);
37 }
```